

Hibernate

Introduction

`spring.jpa.hibernate.ddl-auto` is a Spring Boot property that controls how Hibernate manages the **Database Schema** based on the application's JPA Entities.

The main values are:

- `validate`
- `update`
- `create`
- `create-drop`
- `none`

1. `validate`

`validate` checks whether the existing Database Schema matches the application's Entities. It **does not create, modify, or delete** tables or columns. When the application starts, Hibernate compares the Entity definitions with the existing Database Schema. If there is a mismatch, Hibernate reports an error and the application may fail to start.

Simple Example

```
spring.jpa.hibernate.ddl-auto=validate
```

Suppose we have:

```
@Entity
public class Student {
    @Id
    private Long id;
    private String name;
}
```

Hibernate checks whether the `student` table exists and whether it has the required columns.

If the `name` column is missing, Hibernate will report a schema validation error instead of creating the column.

Real-World Case: `validate` is useful in **Production** when the Database Schema is managed separately using tools such as **Flyway** or **Liquibase**. Hibernate only checks that the schema is correct; it does not change the Production Database.

2. update

update tries to modify the existing Database Schema so that it matches the application's Entities. It attempts to preserve existing data rather than completely recreating the schema. Hibernate compares the Entities with the existing Database Schema. For example, if the Entity contains a new field that does not exist as a column, Hibernate may add that column.

Simple Example

```
spring.jpa.hibernate.ddl-auto=update
```

Initially:

```
@Entity
public class Student {
    @Id
    private Long id;
    private String name;
}
```

The Database may contain:

```
student
-----
id
name
```

Then we add:

```
private String email;
```

When the application starts again, Hibernate may add the email column:

```
student
-----
id
name
email
```

Real-World Case

update is commonly useful in:

- Development environments
- Small projects
- Learning projects
- Prototypes

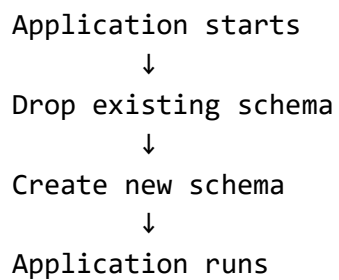
It allows developers to change Entities without manually recreating the Database every time.

However, it is generally **not recommended for serious Production environments**, because schema changes should normally be controlled through versioned migration scripts.

3. create

create tells Hibernate to **drop the existing Database Schema and create a new one** based on the current Entities when the application starts. This means that existing data can be lost.

When the application starts:



Simple Example

`spring.jpa.hibernate.ddl-auto=create`

If we have:

```
@Entity
public class Product {
    @Id
    private Long id;
    private String name;
}
```

Hibernate creates the required product table based on this Entity.

If the application starts again, the existing schema is recreated.

Real-World Case

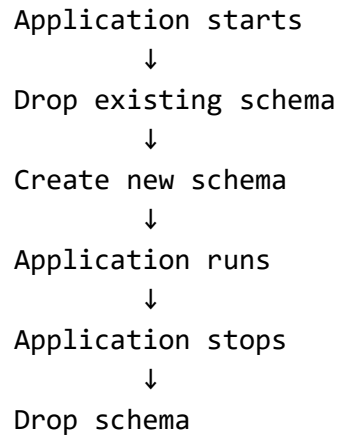
create can be useful for:

- Development
- Testing
- Demo applications
- Temporary databases

It should **not** normally be used in Production because restarting the application can result in losing existing data.

4. create-drop

create-drop creates the Database Schema when the application starts and drops it when the application stops.



Simple Example

`spring.jpa.hibernate.ddl-auto=create-drop`

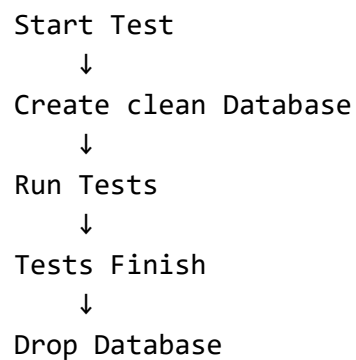
Hibernate creates the required tables when the application starts.

When the application shuts down, Hibernate removes those tables.

Real-World Case

create-drop is particularly useful for **automated testing** and temporary environments.

For example, during integration testing:



This ensures that every test run can start with a clean Database.

It should not be used for Production databases.

5. none

none tells Hibernate **not to manage the Database Schema at all**.

Hibernate will not:

- Create tables
- Update tables
- Drop tables
- Validate the schema

The Database Schema must already exist and must be managed by another system or manually.

Simple Example

```
spring.jpa.hibernate.ddl-auto=none
```

If we have:

```
@Entity
public class Student {
    @Id
    private Long id;
    private String name;
}
```

Hibernate will not create the student table.

The table must already exist in the Database.

Real-World Case

none is useful when Database changes are managed by:

- Flyway
- Liquibase
- SQL migration scripts
- A Database Administration team

This gives the development/DevOps team complete control over Database changes.

Example: `spring.jpa.hibernate.ddl-auto=update`

If we have:

```
spring.jpa.hibernate.ddl-auto=update
```

Hibernate will compare the application's Entities with the existing Database Schema.

For example, if the Entity is:

```
@Entity
public class Employee {

    @Id
    private Long id;
    private String name;
    private double salary;
}
```

And the Database currently has:

```
employee
-----
id
name
```

Hibernate notices that the Entity contains a `salary` field that does not exist in the Database.

It may update the table to:

```
employee
-----
id
name
salary
```

Real-World Case for update

A common use case is **Development**.

During development, developers frequently change their Entities. Using `update` allows Hibernate to automatically apply many schema changes without manually writing SQL for every small change.

For example:

Developer changes Entity
↓
Hibernate detects the change
↓
Hibernate updates the schema
↓
Developer continues working

However, in Production, it is better to use a controlled migration system such as Flyway or Liquibase rather than relying on update.

Comparison Table

Value	What does it do?	Real-World Case
validate	Checks that the Database Schema matches the Entities without changing it.	Production environments where migrations are managed by Flyway/Liquibase.
update	Updates the existing Database Schema to match the Entities while attempting to preserve existing data.	Development, learning projects, and prototypes.
create	Drops the existing schema and creates a new schema when the application starts.	Development and testing when existing data is not important.
create-drop	Creates the schema when the application starts and drops it when the application stops.	Automated integration tests and temporary environments.
none	Does not create, update, drop, or validate the Database Schema.	Production environments where the schema is completely managed by migration tools or DBAs.